

Flask Day 11

Login & Session Management

Student Notes System (SNS) · Python Full Stack Development

Credentials

User lookup

bcrypt verify

Session

Dashboard

Trainer

Madhu Parvathaneni, CTO - Codegnan
madhu@codegnan.com - 8121289993

From Day 10 to Day 11



Today we move from account verification to authenticated access.

Day 10 completed

Registration produced an account, generated OTP, emailed it, verified the OTP and activated the account.

Day 11 focus

Allow verified users to log in, create a session, access dashboard and logout safely.

Trainer check

Ask students: Why should an unverified user not log in even if the password is correct?

Register

Verify OTP

Login

Session

Logout

Learning outcomes

By the end of class, students should explain and implement the login lifecycle.

Create GET and POST login routes

Read and normalize email/password from request.form

Fetch a user using a parameterized MySQL query

Verify password using `bcrypt.check_password_hash()`

Block unverified accounts from logging in

Create and clear Flask sessions

Protect dashboard and implement logout



Today's product milestone

A verified student can log in, access a protected dashboard, and logout. Invalid users, wrong passwords and unverified users are rejected with safe messages.

Evidence to collect

Screenshots for successful login, wrong password, unverified account, dashboard access and logout.

Why do we need login?

Authentication answers: Who is using the application?



Without login

Every visitor looks the same. The application cannot know whose notes to show or protect.

With login

The user proves identity with credentials. Flask stores a session identifier so future requests can be linked to that user.

In SNS

A logged-in user should see only their own notes, files and dashboard information.

Unknown visitor

Login form

Verified identity

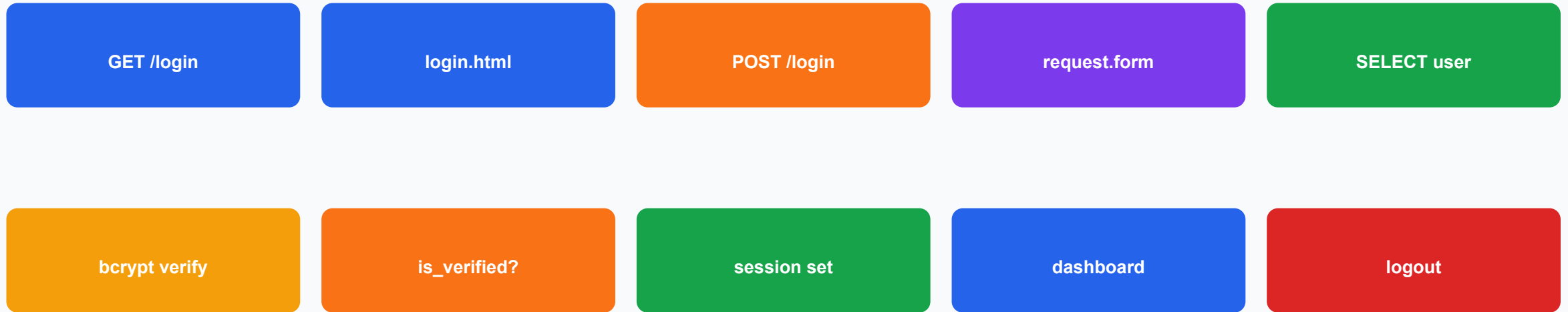
User session

Private dashboard

Day 11 request flow



This is the whiteboard diagram students must reproduce.



GET displays the login form

POST processes submitted credentials

Database lookup uses normalized email

bcrypt checks candidate password against stored hash

Session stores only safe identity values

Users table requirements

Login depends on data created during registration and OTP verification.



```
SELECT id, name, email, password_hash,  
is_verified  
FROM users  
WHERE email = %s;
```

Required columns

id identifies the user. email is used for lookup. password_hash is used for bcrypt verification. is_verified blocks accounts that have not completed OTP.

Teaching point

The login route should never compare plain text passwords and should never reveal whether an email exists.

Safe error

Use a generic message: Invalid email or password. This avoids user enumeration.

Login form essentials



Flask can read only submitted controls that have a name attribute.

```
1 <form method="POST" action="{{ url_for('login') }}">
2   <input type="email" name="email" required>
3   <input type="password" name="password" required>
4   <button type="submit">Login</button>
5 </form>
```

method="POST" sends credentials in request body

name="email" maps to request.form.get("email")

name="password" maps to request.form.get("password")

action can be omitted when posting to same route, but url_for is clearer

Common mistake

Using only id="email" without name="email" means Flask receives no email field.

One route, two responsibilities



GET displays the form; POST processes credentials.

```
1 @app.route('/login', methods=['GET', 'POST'])
2 def login():
3     if request.method == 'GET':
4         return render_template('login.html')
5
6     email = request.form.get('email', '').strip().lower()
7     password = request.form.get('password', '')
8
9     # lookup, verify, session, redirect
```

GET

No credentials yet. Return login.html.

POST

Credentials submitted. Validate and authenticate.

Normalize

`strip()` removes extra spaces. `lower()` avoids email case mismatches.

Helper: find user by email

Keep database lookup separate from route logic.



```
1 def find_user_by_email(email):
2     connection = get_db_connection()
3     cursor = connection.cursor(dictionary=True)
4
5     cursor.execute(
6         """
7         SELECT id, name, email, password_hash, is_verified
8         FROM users
9         WHERE email = %s
10        """,
11        (email,)
12    )
13
14    user = cursor.fetchone()
15    cursor.close()
16    connection.close()
17    return user
```

dictionary=True makes user["name"] readable

fetchone() returns one user or None

%s placeholder keeps SQL and user input separate

(email,) must be a tuple with one value

Debug question

What happens when the email is not found? user becomes None.

Password verification with bcrypt



We never decrypt a password. We verify a candidate against a stored hash.

Entered password

Stored hash

bcrypt check

True / False

```
1 valid_password = bcrypt.check_password_hash(  
2     user['password_hash'],  
3     password  
4 )  
5  
6 if not valid_password:  
7     flash('Invalid email or password.', 'danger')  
8     return render_template('login.html'), 401
```

Do not compare password == password_hash

Do not store plain password

Same password can produce different hashes

check_password_hash returns True or False

Account verification gate

A correct password is not enough if OTP verification is incomplete.

```
1 if not user or not bcrypt.check_password_hash(
2     user['password_hash'], password):
3     flash('Invalid email or password.', 'danger')
4     return render_template('login.html'), 401
5
6 if not user['is_verified']:
7     flash('Please verify your account first.',
8         'warning')
9     return redirect(url_for('verify_otp', email=email))
```



Case 1

Unknown email or wrong password → generic invalid message.

Case 2

Correct credentials but not verified → redirect to OTP verification.

Case 3

Correct credentials and verified → create session.

What is a session?



A session lets Flask remember a logged-in user across requests.

Login success

`session["user_id"]`

Browser cookie

Next request

Dashboard

Important idea

HTTP is stateless. Without a session, every request is independent and Flask cannot know who is logged in.

What to store

Store small identity values such as `user_id` and `user_name`. Do not store password or sensitive secrets.

```
1 session.clear()
2 session['user_id'] = user['id']
3 session['user_name'] = user['name']
```

SECRET_KEY is required for sessions

Flask uses SECRET_KEY to sign session cookies and flash messages.



```
1 import os
2
3 app = Flask(__name__)
4 app.config['SECRET_KEY'] = os.getenv(
5     'SECRET_KEY',
6     'dev-secret-change-me'
7 )
```

Classroom use

A temporary development secret is acceptable for local practice.

Project rule

Do not commit real secrets. Use .env and keep it in .gitignore.

Symptom

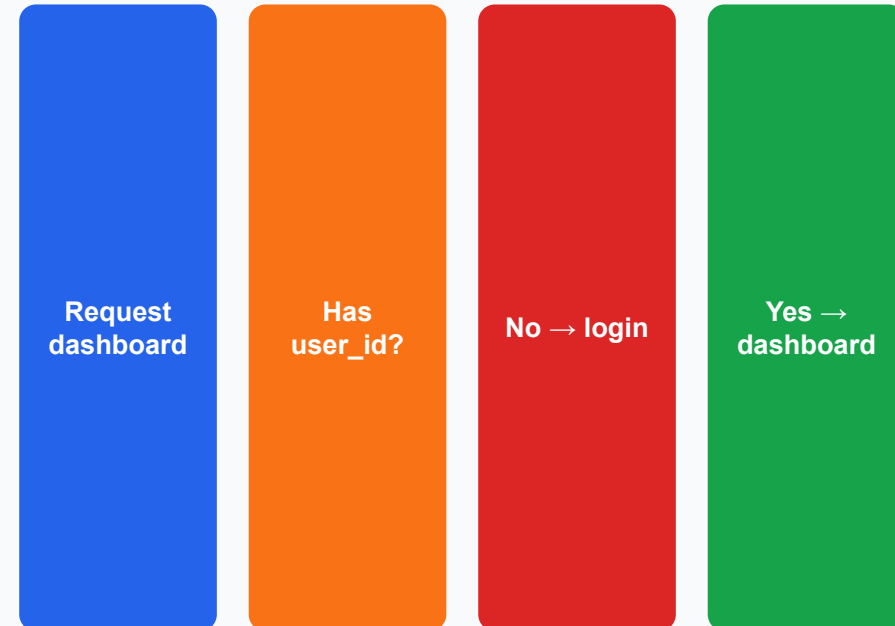
If SECRET_KEY is missing, sessions and flash messages may fail.

Protecting the dashboard

A private page must check whether the user is logged in.



```
1 @app.get('/dashboard')
2 def dashboard():
3     if 'user_id' not in session:
4         flash('Please login first.', 'warning')
5         return redirect(url_for('login'))
6
7     return render_template(
8         'dashboard.html',
9         username=session.get('user_name')
10    )
```



Acceptance test

Open /dashboard in a new incognito window. It should redirect to /login.

Reusable login_required decorator

Avoid repeating the same session check in every private route.



```
1 from functools import wraps
2
3 def login_required(view_func):
4     @wraps(view_func)
5     def wrapper(*args, **kwargs):
6         if 'user_id' not in session:
7             flash('Please login first.', 'warning')
8             return redirect(url_for('login'))
9         return view_func(*args, **kwargs)
10    return wrapper
```

```
1 @app.get('/dashboard')
2 @login_required
3 def dashboard():
4     return
render_template('dashboard.html')
```

Decorator runs before the route function

If not logged in, redirect to login

If logged in, allow original function to run

Logout clears the session



Logout removes the remembered identity from the browser session.

```
1 @app.get('/logout')
2 def logout():
3     session.clear()
4     flash('Logged out successfully.',
5         'success')
6     return redirect(url_for('login'))
```

Click logout

`session.clear()`

Redirect login

Dashboard blocked

Test it

Login → open dashboard → logout → try /dashboard again. It must redirect to login.

Flash messages for user feedback



Use flash to show safe success or error messages after redirects.

```
1 from flask import flash
2
3 flash('Invalid email or password.', 'danger')
4 flash('Please login first.', 'warning')
5 flash('Logged out successfully.', 'success')
```

```
1 {% with messages =
get_flashed_messages(with_categories=True) %}
2   {% for category, message in messages %}
3     <div class="alert alert-{{ category }}">{{
message }}</div>
4   {% endfor %}
5 {% endwith %}
```

Placement

Put the flash rendering block in base.html so every page can display messages.

Security

For failed login, use generic messages. Do not say “email exists but password wrong.”

Reference implementation: login route



Use this as the final combined version after incremental build.

```
1 @app.route('/login', methods=['GET', 'POST'])
2 def login():
3     if request.method == 'GET':
4         return render_template('login.html')
5
6     email = request.form.get('email', '').strip().lower()
7     password = request.form.get('password', '')
8     user = find_user_by_email(email)
9
10    valid = user and user['is_verified'] and bcrypt.check_password_hash(
11        user['password_hash'], password
12    )
13
14    if not valid:
15        flash('Invalid email or password.', 'danger')
16        return render_template('login.html'), 401
17
18    session.clear()
19    session['user_id'] = user['id']
20    session['user_name'] = user['name']
21    return redirect(url_for('dashboard'))
```

Common security mistakes



Use this slide as a warning before students implement independently.

Mistake 1

Comparing raw password with password_hash.

Mistake 2

Creating a session before password verification.

Mistake 3

Allowing unverified users to log in.

Mistake 4

Storing password or full user record in session.

Mistake 5

Using different error messages that reveal whether an email exists.

Mistake 6

Forgetting to protect dashboard and private routes.

Rule to remember

Authenticate first. Create session only after successful verification. Protect every private route.

Testing matrix



Day 11 is not complete until both success and failure paths are tested.

Test case	Expected result
Valid verified user	Redirects to dashboard
Wrong password	Shows generic invalid message
Unknown email	Shows generic invalid message
Unverified account	Redirects to OTP verification
Direct /dashboard without login	Redirects to login
Logout then /dashboard	Redirects to login

Trainer-led debugging drills



Introduce one defect at a time and make students diagnose the layer.

Bug 1

Login always fails → inspect form field names and request.form values.

Bug 2

Correct password rejected → check password_hash column and bcrypt call order.

Bug 3

Dashboard accessible without login → missing session guard.

Bug 4

Flash messages not shown → base.html missing get_flashed_messages block.

Bug 5

Session not working → missing or unstable SECRET_KEY.

Bug 6

Logout appears to work but dashboard opens → browser still has session or route not protected.

Guided lab: Day 11 milestone



Students reproduce the feature, test it, then commit evidence.

Update imports

Add SECRET_KEY

Build login form

Add helper

Verify bcrypt

Create session

Protect dashboard

Add logout

Show flash

Test matrix

Implement the trainer version first

Add one independent improvement: better message, reusable helper, or additional test case

Capture evidence for successful login and two failures

Update README with new routes and session notes

Commit: Day 11: implement login and session management

Day 11 viva questions

Use these as exit-ticket questions before ending class.

What is the difference between authentication and authorization?

Why do we use `bcrypt.check_password_hash()`?

Why should the login error message be generic?

What does `session.clear()` do?

What values are safe to store in session?

Why does `/dashboard` need a session check?

What happens if `SECRET_KEY` is missing?

How do you test logout properly?



Expected answer style

Students must answer using request-flow language:
browser → Flask route → database → bcrypt → session → response.

Trainer check

Ask one student to draw the full login flow without seeing code.

Day 11 recap

Students now have authenticated access in SNS.

Login form submits email and password using POST

Flask reads credentials from request.form

MySQL fetches user by normalized email

bcrypt verifies the candidate password

Only verified users receive a session

Dashboard is protected using session checks

Logout clears the session and redirects to login



Next class: Day 12

Forgot Password: email lookup, reset OTP/token, password update query and secure reset flow.

Login

Session

Protected
Dashboard

Logout